

Notes on Apriori and Eclat

Luis Mendez

ASSOCIATION RULE LEARNING

Apriori and Eclat are both association rule learning algorithms, they are unsupervised, we are not predicting a target variable, instead we are looking for relationships between items that co-occur together frequently in a dataset. The classic example, and the one used in the `Market_Basket_Optimisation.csv` dataset, is market basket analysis: given 7501 customer transactions, each containing up to 20 purchased products, we want to find rules like “customers who buy product A also tend to buy product B”. These rules can then be used for things like product placement or recommendation systems (people who bought this also bought that).

Both algorithms rely on the same three core metrics:

- **Support:** how frequently an itemset appears in the dataset.

$$\text{support}(A) = \frac{\text{number of transactions containing } A}{\text{total number of transactions}}$$

- **Confidence:** given that a transaction contains A , how likely is it to also contain B .

$$\text{confidence}(A \rightarrow B) = \frac{\text{support}(A \cup B)}{\text{support}(A)}$$

- **Lift:** how much more likely B is to be bought given A , compared to B being bought on its own. A lift greater than 1 means A and B are positively correlated, a lift of 1 means they are independent, and less than 1 means they are negatively correlated (buying A makes B less likely).

$$\text{lift}(A \rightarrow B) = \frac{\text{confidence}(A \rightarrow B)}{\text{support}(B)}$$

APRIORI

Apriori builds association rules of the form $A \rightarrow B$ (if a customer buys A , they are likely to also buy B). It works by growing frequent itemsets level by level, first finding all frequent single items, then combining them into pairs, then triples, and so on, at every step pruning candidate itemsets whose support is below a minimum threshold (the “apriori principle”: if an itemset is infrequent, all of its supersets must also be infrequent, so there is no need to check them).

In practice, using the `apriori` library:

- We set `min_support`, `min_confidence`, and `min_lift` to filter out weak or irrelevant rules. For the market basket dataset a support of 0.003 roughly means an item needs to show up in about 3 transactions a day over the week the data was collected to be considered.

- We also set `min_length` and `max_length` to 2, so that we only look at rules involving exactly two products (one on the left hand side, one on the right hand side), which keeps the rules simple and interpretable.
- The output rules include the left hand side item, the right hand side item, the support, the confidence, and the lift of the rule, so that the results can be sorted (e.g., by descending lift) to surface the strongest and most interesting associations first.

Apriori is more computationally expensive than Eclat because it has to compute support, confidence, and lift for every candidate rule, and it needs to keep track of the direction of the rule ($A \rightarrow B$ is not the same as $B \rightarrow A$).

ECLAT

Eclat (Equivalence Class Transformation) simplifies the problem: instead of looking for directional rules with confidence and lift, it only looks for frequent itemsets and ranks them by support alone. It does not distinguish a left hand side from a right hand side, an Eclat result of the form $\{A, B\}$ just means that A and B are frequently bought together, without saying which one implies the other.

Under the hood, Eclat represents each item by the set (or a vertical bit-vector) of transactions that contain it, and computes the support of a combined itemset by intersecting these transaction sets, rather than repeatedly scanning the whole dataset horizontally the way Apriori does. This tends to make Eclat faster for finding frequent itemsets, since it avoids recomputing confidence/lift and avoids generating both directions of every rule.

Because it only reports support, Eclat’s output is much simpler: just the pair (or itemset) of products and how often they occur together, sorted by descending support to find the itemsets that appear most frequently.

WORKED EXAMPLE: MARKET BASKET OPTIMISATION

Running `apriori` on the 7501 transactions with `min_support = 0.003`, `min_confidence = 0.2`, `min_lift = 3`, and rules restricted to exactly two items (`min_length = max_length = 2`) yields the same 9 frequent, sufficiently-confident, positively-correlated item pairs for both algorithms, since Eclat here is implemented as the same `apriori()` call with confidence and lift discarded from the output.

Apriori, sorted by descending lift, surfaces the strongest implications first:

LHS	RHS	Support	Confidence	Lift
fromage blanc	honey	0.0033	0.245	5.16
light cream	chicken	0.0045	0.291	4.84
pasta	escalope	0.0059	0.373	4.70

`fromage blanc` $\rightarrow$ `honey` has the lowest support of the nine rules (it is a rarer pairing overall) but the highest lift: when a customer does buy `fromage blanc`, they are over $5\times$ more likely than average to also buy `honey`, i.e., a strong, if infrequent, implication.

Eclat, sorted by descending support, instead surfaces the most common co-occurrences regardless of implication strength:

Product 1	Product 2	Support
herb & pepper	ground beef	0.0160
whole wheat pasta	olive oil	0.0080
pasta	escalope	0.0059

`herb & pepper` and `ground beef` appear together roughly $3\times$ more often than any other pair in the filtered set, but Eclat’s output alone does not tell us whether buying one makes the other more likely relative to its baseline popularity, that requires the confidence/lift that only Apriori’s directional output provides (lift 3.29, i.e., still a meaningfully positive association, just not the strongest one).

This is the practical tradeoff in a nutshell: Eclat’s ranking answers “what do people buy together most often”, Apriori’s ranking answers “what pairing is the most surprising/informative given each item’s overall popularity”. For a recommendation shelf near the register, Eclat’s most-frequent pairs are safer bets; for cross-selling or highlighting a non-obvious pairing, Apriori’s highest-lift rules are more useful.

APRIORI VS. ECLAT

- Apriori produces directional rules ($A \rightarrow B$) with support, confidence, and lift, Eclat produces undirected frequent itemsets with only support.
- Apriori tells us about the strength and reliability of an implication (via confidence/lift), Eclat only tells us how often items co-occur.
- Eclat is generally simpler and computationally cheaper since it skips the confidence/lift calculations and does not need to consider both orderings of every pair.
- In practice, when using `apyori` for Eclat, the same underlying algorithm (`apriori()`) is used with the same thresholds, we just discard confidence and lift from the output and keep only support, this is a common simplification/shortcut rather than using a true optimized Eclat implementation.